

레이다 SW 학습 과제 3

표적 궤적 모의

Target Trajectory Simulation

Janghwan Kim (Harley)

INDEX

표적 궤적 모의 TargetSim

Janghwan Kim (Harley)

01

표적 구조체 설계

상태 벡터와 좌표계
기동을 $G \cdot \text{축} \cdot \text{구간}$ 으로

5 ~ 9

02

표적 궤적 모의 구현

운동학 모델과 회전행렬
속도 변환과 중점법 적분

10 ~ 14

03

시뮬레이션 결과 및 검증

명세 조건 60 s 결과
세 갈래 검증과 시연

15 ~ 18

과제가 요구한 것

「과제 3 내용」의 요구사항을 네 덩어리로 묶고, 발표도 그 순서로 갑니다.

1 표적 구조체 정의

표적 최대 10 · 기동 최대 30.
위치는 위도 · 경도 · 고도,
속도는 동체 속도 한 개.
기동은 G · 축 · 시작 · 종료.

01 장

2 표적 상태 갱신

위치와 속도를 ECEF 에서 갱신.
추가 고민 — 동체 속도를 ECEF
 V_x, V_y, V_z 로 바꾸는 방법.

02 장

3 시뮬레이션 조건

60 s · 0.1 s 간격.
플랫폼 1 + 대함 1 + 대공 1.
제공받은 좌표변환 코드 사용.

03 장

4 결과 제시

궤적을 LLA 로 출력하고 그림으로.
구조체 · 구현 · 결과를
코드와 함께 설명.

03 장

표적 모의는 실장비 없이 데이터처리 입력을 공급하는 일입니다 — 같은 규격, 반복 가능, 그리고 내가 넣은 값이 곧 참값입니다.
좌표변환과 행렬 연산은 제공받은 코드를 무수정으로 썼고, 새로 작성한 것은 TargetSim.c 하나입니다.

만든 것 — 계산과 화면을 나눈다

연산 엔진만 따로 빌드해 검증할 수 있게 했습니다. 오늘 숫자는 모두 그렇게 뽑은 것입니다.

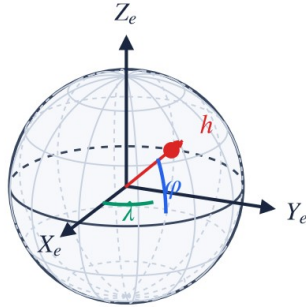


01. 표적 구조체 설계

표적의 상태와 좌표계

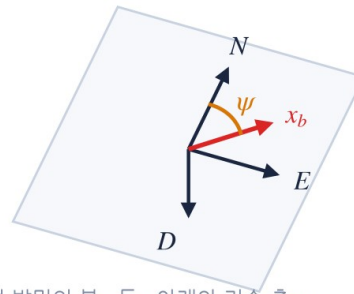
표적 하나는 아홉 개 성분으로 정해집니다. 그 값들이 어느 좌표계의 값인지부터 맞춥니다.

ECEF · LLA



지구중심 직교좌표와 그 각도 표현 (φ, λ, h)

NED · Body



표적 발명의 북·동·아래와 기수 축 x_b

자세각 세 개

Roll ϕ



날개가 기운다

Pitch θ



기수가 들린다

Yaw ψ



기수가 돈다

$$\mathbf{C}_b^n = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$$

곱하는 순서는 Yaw $\rightarrow$ Pitch $\rightarrow$ Roll 로 고정한다

표적 하나의 상태 벡터

$$\mathbf{x}(t) = [\mathbf{p}^{eT} \ \mathbf{v}^{eT} \ \boldsymbol{\Theta}^T]^T \in \mathbb{R}^9$$

■	$\mathbf{p}^e$	ECEF 위치	m
■	$\mathbf{v}^e$	ECEF 속도	m/s
■	$\boldsymbol{\Theta}$	자세각 (ϕ, θ, ψ)	rad

명세가 주는 입력이 초기 상태가 되는 길

■	위도 · 경도 · 고도 좌표변환 1회	$(\varphi_0, \lambda_0, h_0)$	$\rightarrow$	$\mathbf{p}_0^e$
■	자세각 그대로	$(\phi_0, \theta_0, \psi_0)$	$\rightarrow$	$\boldsymbol{\Theta}_0$
■	동체 속력 1개 회전 2회 — 이 과제의 추가 고민 항목	V	$\rightarrow$	$\mathbf{v}_0^e = \mathbf{C}_n^e \mathbf{C}_b^n [V \ 0 \ 0]^T$

01. 표적 구조체 설계

기동을 네 개의 값으로

"12 초부터 16 초까지 오른쪽으로 8 G" 라는 지시가 구조체 한 줄이 되고, G 하나가 회전 속도를 정합니다.

기동 1건 = 값 네 개

enTurnType	0 없음 / 1 Roll / 2 Yaw / 3 Pitch
gravityValue	크기는 세기, 부호는 회전 방향 [G]
startTime	이 시각부터 (포함) [s]
endTime	이 시각까지 (제외) [s]

$t \in [t_{start}, t_{end})$ 끝과 다음 시작이 같아도 겹침이 아니다

실제로 겹치면 계산 전에 TGT_ERR_MANEUVER_OVERLAP

G 값 하나가 각속도와 선회 반경을 정한다

$$a_c = \frac{V^2}{R} = V\omega = n g$$

원운동의 구심가속도를 중력가속도의 n 배로 적는다

$$\omega = \frac{n g}{V}, \quad R = \frac{V^2}{n g}$$

$\omega = (\text{gravityValue} * G_FORCE) / \text{headingSpeed};$

n [G]	대함 V = 30 m/s		대공 V = 200 m/s	
1	18.7 °/s	R = 91.8 m	2.8 °/s	R = 4079 m
4	74.9 °/s	R = 22.9 m	11.2 °/s	R = 1020 m
8	149.8 °/s	R = 11.5 m	22.5 °/s	R = 510 m

기동표 6줄이 실제 응답으로 나타난 모습 (대공 표적)



같은 G 라도 빠른 표적은 덜 꺾인다 — 분모에 속력이 있기 때문이다.

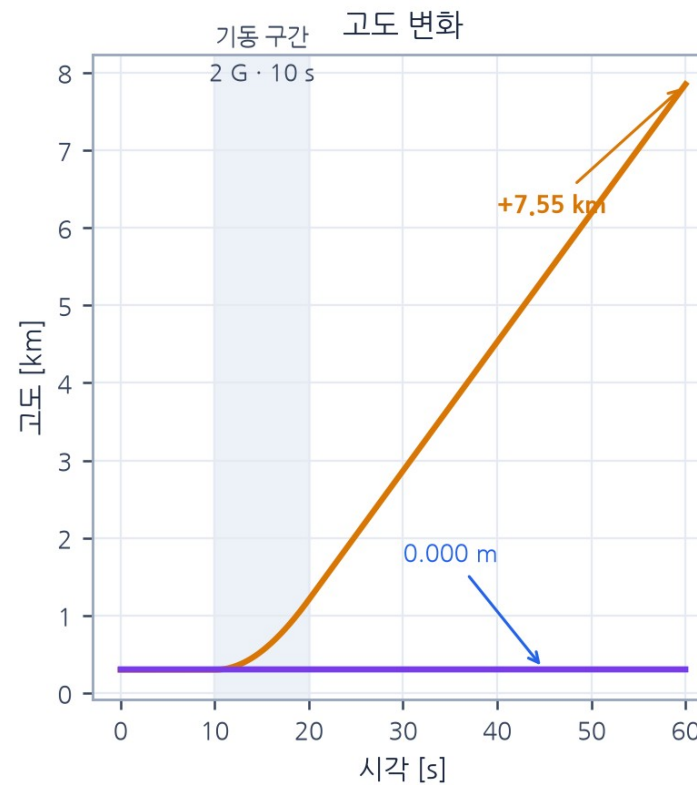
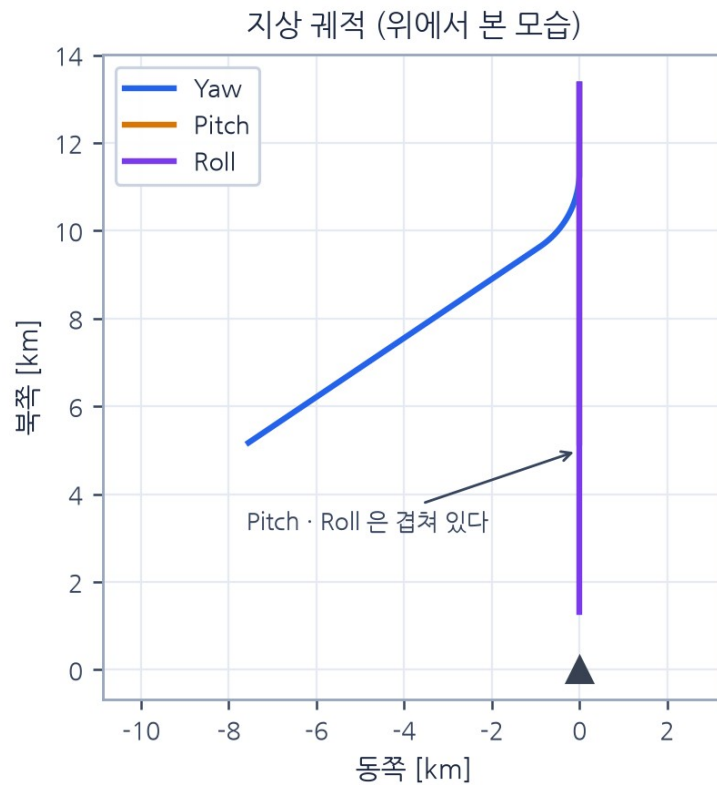
색칠한 구간이 기동표에 적은 [start, end) 이고,
그 안에서만 기울기가 생긴다. 기울기가 곧 ω 다.

기동 지시는 "위도 32.1 에서 좌 90도" 처럼 좌표로 온다.
구조체에는 시각 칸만 있어 Core 를 돌려 그 좌표에 닿는
시각을 찾아 start / end 로 옮겼다.

01. 표적 구조체 설계

회전축 세 가지의 응답 — 실측

같은 표적에 같은 2 G 를 10 초씩, 축만 바꿔 걸어 본 결과입니다.



같은 표적 · 같은 2 G · 같은 10 s, 축만 바꿨다

Yaw

ψ 가 118° 돌고 궤적이 휜다.
고도는 300.000 m 그대로.

Pitch

θ 가 들려 7.55 km 상승.
지상 궤적은 거의 제자리.

Roll

ϕ 만 변하고 궤적은 불변.
회전축이 곧 속도 방향이라 그렇다.

Roll 이 궤적을 못 바꾸는 것은 구현 오류가 아니라
물리적으로 옳은 결과다.

01. 표적 구조체 설계

구조체 관계와 설계 결정

입력 — 시나리오가 채운다

ST_SimConfig			
durationTime	FLOAT64 [s]		
stepTime	FLOAT64 [s]		
st_Platform	ST_PlatformInit	1	
nTargetNum	INT32	1..10	
st_Target[10]	ST_TargetInit	10	

ST_TargetInit			
st_InitLla	STRUCT_Coord_Lla	rad, m	
st_InitAtt	STRUCT_Coord_Attitude	rad	
headingSpeed	FLOAT64 [m/s]	> 0	
nManeuverNum	INT32	0..30	
st_Maneuver[30]	ST_TargetManeuver	30	

ST_TargetManeuver			
enTurnType	EN_TurnType	0..3	
gravityValue	FLOAT64 [G]	±	
startTime / endTime	FLOAT64 [s]	[a, b)	

→
f_Tgt_InitSim / f_Tgt_StepSim

출력 — Core 가 채운다

ST_SimState			
nStepNum	INT32	600	
st_Sample	ST_SimSample	최근 1	
st_Config	ST_SimConfig	설정 사본	
설정을 복사해 두어 호출 중 원본이 바뀌어도 진행이 흔들리지 않는다			

ST_SimSample			
simTime	FLOAT64 [s]		
nStepIndex	INT32	0..600	
st_Platform	ST_TargetState	1	
st_Target[10]	ST_TargetState	10	

ST_TargetState			
st_PosEcef	STRUCT_Coord_Rect [m]	갱신	
st_VelEcef	STRUCT_Coord_Rect [m/s]	갱신	
st_Lla	STRUCT_Coord_Lla	출력용	
st_Att	STRUCT_Coord_Attitude	갱신	

설계 결정 세 가지

- 입력 / 출력 분리
- 고정 배열
- 제공 구조체 재사용

설정은 한 번만 읽고, 스텝마다 바뀌는 것은 상태뿐이다
 10 × 30 을 배열로 박아 실행 중 메모리 할당이 실패할 자리를 없앴다
 STRUCT_Coord_Lla / _Attitude 는 좌표변환 헤더의 것을 그대로 쓴다

01. 표적 구조체 설계

코드로 보는 구조체

입력 — TargetSim.h

```
// 기동 1 건 . gravityValue 의 부호가 회전 방향이다 .
typedef struct
{
    EN_TurnType    enTurnType;    // 0~3
    FLOAT64        gravityValue;  // [G]
    FLOAT64        startTime;     // [s]
    FLOAT64        endTime;       // [s]
} ST_TargetManeuver;

// 위치는 위 / 경 / 고도 , 속도는 동체 x 축 방향 크기 .
typedef struct
{
    STRUCT_Coord_Lla    st_InitLla;
    STRUCT_Coord_Attitude st_InitAtt;
    FLOAT64              headingSpeed;  // V_heading
    INT32                nManeuverNum;
    ST_TargetManeuver     st_Maneuver[TGT_MAX_MANEUVER_NUM];
} ST_TargetInit;
```

출력 — TargetSim.h

```
// 위치 · 속도는 ECEF 에서 갱신하고 , LLA 는 출력과
// 다음 스텝의 NED 기준으로 함께 보관한다 .
typedef struct
{
    FLOAT64        simTime;    // [s]
    STRUCT_Coord_Rect st_PosEcef;  // [m]
    STRUCT_Coord_Rect st_VelEcef;  // [m/s]
    STRUCT_Coord_Lla st_Lla;
    STRUCT_Coord_Attitude st_Att;
} ST_TargetState;

// 한 시각의 플랫폼 · 표적 상태 .
typedef struct
{
    FLOAT64        simTime;
    INT32          nStepIndex;
    ST_TargetState st_Platform;
    ST_TargetState st_Target[TGT_MAX_TARGET_NUM];
} ST_SimSample;
```

구간은 [시작, 종료)

끝과 다음 시작이 같아도 겹침이 아니다.
실제로 겹치면 계산 전에 거부한다.

ECEF 와 LLA 를 함께

갱신은 ECEF, 출력은 LLA, 다음 스텝의
NED 기준도 LLA — 한 번만 변환해 둔다.

플랫폼도 같은 구조체

기동이 없는 표적일 뿐이라 각속도만 0 으로
넣어 같은 경로로 전진시킨다.

02. 표적 궤적 모의 구현

운동학 모델과 ECEF 적분

구현을 시작하기 전에 방정식부터 세우고, 어느 좌표계에서 적분할지를 정합니다.

표적 운동을 지배하는 세 줄

$$\dot{\mathbf{p}}^e = \mathbf{v}^e$$

위치의 시간 변화율이 곧 속도

$$\mathbf{v}^e = \mathbf{C}_n^e(\varphi, \lambda) \mathbf{C}_b^n(\phi, \theta, \psi) [V \ 0 \ 0]^T$$

속도는 기수 방향 크기 V 를 두 번 돌려 만든다

$$\dot{\boldsymbol{\Theta}} = \boldsymbol{\omega}(t), \quad \omega_i = n_i g / V$$

자세각은 기동이 걸린 구간에서만 돈다

이 모델이 전제하는 것

운동학 모델 힘이 아니라 속도 · 각속도를 직접 준다.
중력 · 추력 · 항력을 풀지 않아 탄도 표적에는 맞지 않는다.

속력 일정 회전만 하므로 $|v|$ 가 변하지 않는다.
가속이 필요하면 V 를 시간 함수로 바꿔야 한다.

오일러각 적분 자세각을 직접 적분한다.
 $\theta \rightarrow \pm 90^\circ$ 에서 짐벌락 — 사원수로 바꾸면 풀린다.

왜 ECEF 에서 적분하는가

LLA 에서 직접

$$\dot{\varphi} = \frac{v_N}{M(\varphi) + h}$$

분모가 위도마다 달라
곧게 날려도 고도가 뜬다



ECEF 에서

$$\mathbf{p}_{k+1}^e = \mathbf{p}_k^e + \mathbf{v}^e \Delta t$$

미터 단위 직교좌표라
그냥 더하면 된다

본 과제 조건에서 실제로 얼마인가

$$d = V \Delta t$$

$$20 \text{ m}$$

$$d^2/2R$$

$$31.4 \text{ } \mu\text{m}/\text{스텝}$$

$$600 \text{ 스텝 누적}$$

$$18.8 \text{ mm}$$

$$\text{중점법 보정 후}$$

$$\text{고도 불변}$$

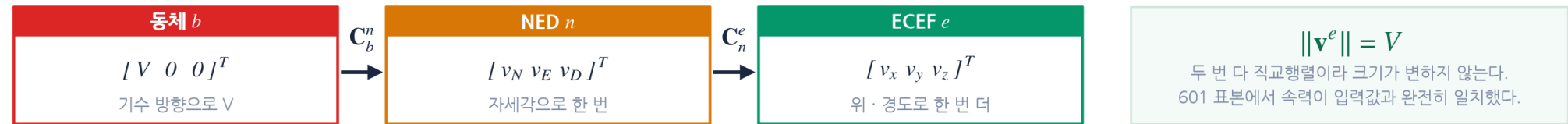
값은 작지만, 이런 항이 남으면 간격을 줄여도 오차가 선형으로만 줄어 2차 수렴이 깨진다.

02. 표적 궤적 모의 구현

동체 속도를 ECEF 속도로 — 과제의 추가 고민

" 동체 속도 값 (V_heading) 을 ECEF 좌표계 V_x, V_y, V_z 성분으로 변환하는 방법 " 에 대한 답입니다 .

동체 속도를 ECEF 속도로 — 회전 두 번



① 동체 → NED

$$C_b^n(\phi, \theta, \psi)$$

$$\begin{bmatrix} c\psi c\theta & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix}$$

ϕ Roll θ Pitch ψ Yaw ($c = \cos$, $s = \sin$)

쓰는 방법이 위치와 속도에서 다르다

$$\mathbf{p}^e = C_n^e \mathbf{p}^n + \mathbf{p}_{ref}^e$$

위치는 기준점을 더한다

$$\mathbf{v}^e = C_n^e \mathbf{v}^n$$

② NED → ECEF

$$C_n^e(L, \lambda)$$

$$\begin{bmatrix} -sL c\lambda & -s\lambda & -cL c\lambda \\ -sL s\lambda & c\lambda & -cL s\lambda \\ cL & 0 & -sL \end{bmatrix}$$

L 위도 λ 경도 — 표적이 움직이면 매 스텝 다시 만든다

속도는 방향뿐이라 기준점을 0 으로 준다 — `f_Trans_Ned_To_Ecef(..., 0.0, 0.0, 0.0, ...)`

02. 표적 궤적 모의 구현

코드 ① — 동체 속도를 ECEF 속도로

TargetSimCore/src/TargetSim.c

```
// 동체 속도 (V, 0, 0) 을 자세로 한 번 , 위경도로 한 번 돌려 ECEF 속도로 만든다 .  
// 회전만 거치므로 속력은 그대로 보존된다 .  
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,  
    const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)  
{  
    STRUCT_Coord_Rect st_VelNed;  
  
    // 참고 소스의 f_Trans_Body_To_Ned 는 인자를 Roll, Yaw, Pitch 순서로 받는다 .  
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,  
        st_Att->Roll, st_Att->Yaw, st_Att->Pitch);  
  
    // f_Trans_Ned_To_Ecef 는 기준점 이동까지 더한다 . 방향만 옮길 때는 기준점을 원점으로 준다 .  
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,  
        0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);  
}
```

기준점을 0 으로 주는 이유

f_Trans_Ned_To_Ecef 는 회전에 더해
기준점만큼 평행이동까지 한다 . 속도는
" 위치 " 가 아니라 " 방향 " 이라 빼야 한다 .

인자 순서를 주석에 남긴 이유

제공받은 함수가 Roll, Yaw, Pitch 순서로
받는다 . 자세각 구조체 필드 순서와 달라
한 번 틀렸던 자리다 .

속력 보존은 성질 , 시험은 확인

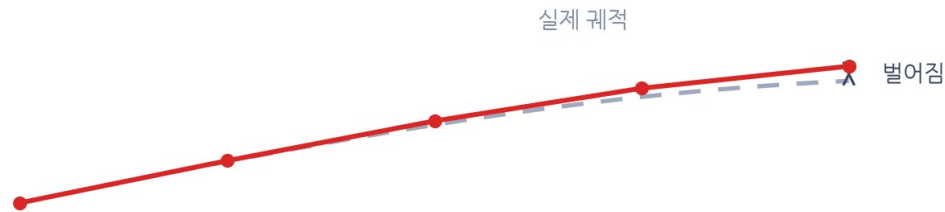
회전행렬은 직교행렬이라 크기를 바꾸지
않는다 . 601 표본에서 속력이 소수점
12 자리까지 입력값과 일치했다 .

02. 표적 궤적 모의 구현

0.1 초를 어떻게 전진시킬 것인가

어느 지점의 기울기를 쓰느냐가 오차 차수를 가릅니다.

오일러법 — 출발할 때의 기울기로 한 스텝 간다



중점법 — 반 스텝 가서 기울기를 다시 재고 한 스텝 간다



굽은 길을 직선 토막으로 잇는 것은 같다. 다른 것은 어느 지점의 기울기를 쓰느냐다 — 중점의 기울기가 구간 평균에 훨씬 가깝다.

두 방법의 식과 오차 차수

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f(t_k, \mathbf{x}_k)$$

국소 $O(\Delta t^2)$ · 전역 $O(\Delta t)$ — 1차

$$\mathbf{k}_l = f(t_k, \mathbf{x}_k)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f\left(t_k + \frac{\Delta t}{2}, \mathbf{x}_k + \frac{\Delta t}{2} \mathbf{k}_l\right)$$

국소 $O(\Delta t^3)$ · 전역 $O(\Delta t^2)$ — 2차

이 과제에서의 반 스텝

자세각	각속도가 상수라 반 스텝 값을 바로 더한다
위치	스텝 시작 속도로 반 스텝 예측한다
속도	그 자리 · 그 자세에서 다시 만든다

계산량은 좌표변환 2회 → 4회로 늘지만, 0.1 s 에서 오차가 1.36 m → 1.36 cm 로 100 배 줄었다.

02. 표적 궤적 모의 구현

코드 ② — 한 스텝 전진 (중점법)

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발췌)

```
halfStep = 0.5 * stepTime;

// ① 반 스텝 뒤의 자세 — 각속도가 구간 내 상수라 바로 더한다
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 — 스텝 시작 속도로 예측
st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep);
...

// ③ 그 자리 · 그 자세에서 구한 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
...
```

② 를 빼먹었을 때 생긴 일

자세만 반 스텝 돌리고 위치를 그대로 두면,
표적은 앞으로 나아갔는데 "북쪽"의 방향은
뒤에 남는다. 그 차이가 매 스텝 $d^2/2R$ 만큼
고도로 쌓인다.

계산이 깨져도 상태는 지킨다

사본에서 전부 계산하고 유효성을 확인한 뒤
모두 성공했을 때만 한 번에 반영한다.

알고도 남겨 둔 한계 — 짐벌락

Pitch 가 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과
겹쳐 Yaw 기동이 궤적을 바꾸지 못한다.

03. 시뮬레이션 결과 및 검증

시뮬레이션 조건

「과제 3 내용」 2 항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270 (서)
대공 표적	32.12	126.0	300	200	0	0	180 (남)

시간 60 s · 간격 0.1 s

스텝 600 회, 표본 601 개.
시각은 누적이 아니라 스텝 번호 × 간격으로
매겨 받을림 오차가 쌓이지 않게 했다.

검증하기 좋은 조건이다

기동이 없으므로 답을 미리 알 수 있다.
대함은 정서진 → 위도 불변,
대공은 정남진 → 경도 불변, 둘 다 고도 불변.

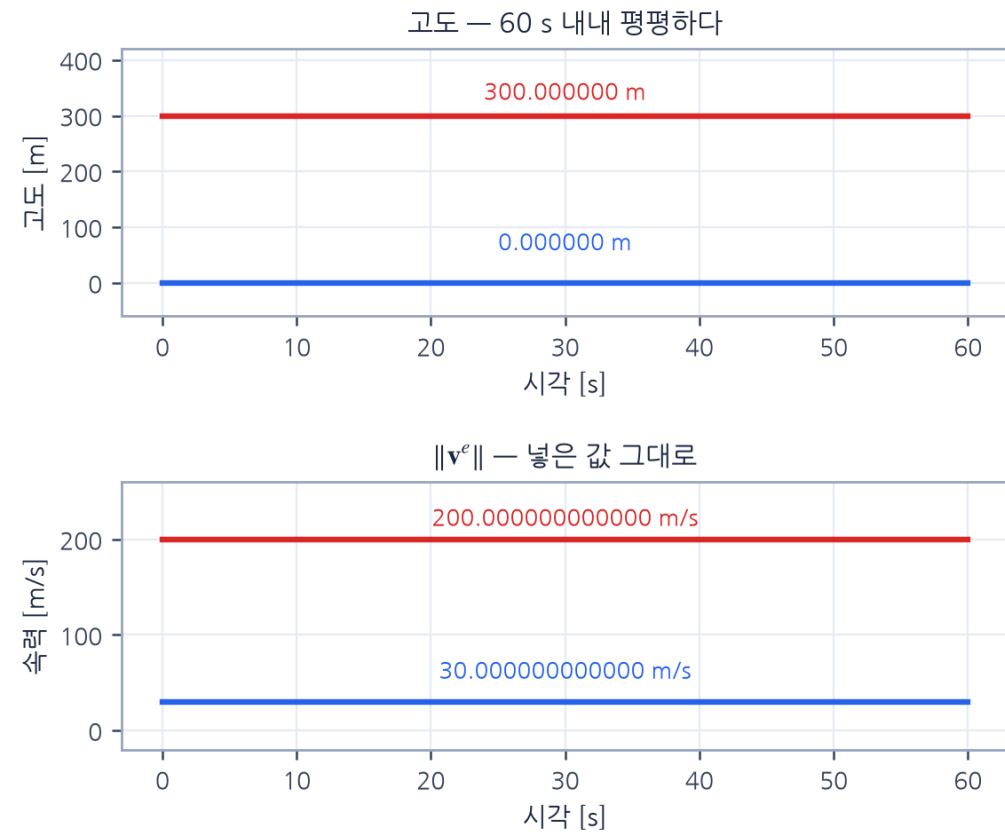
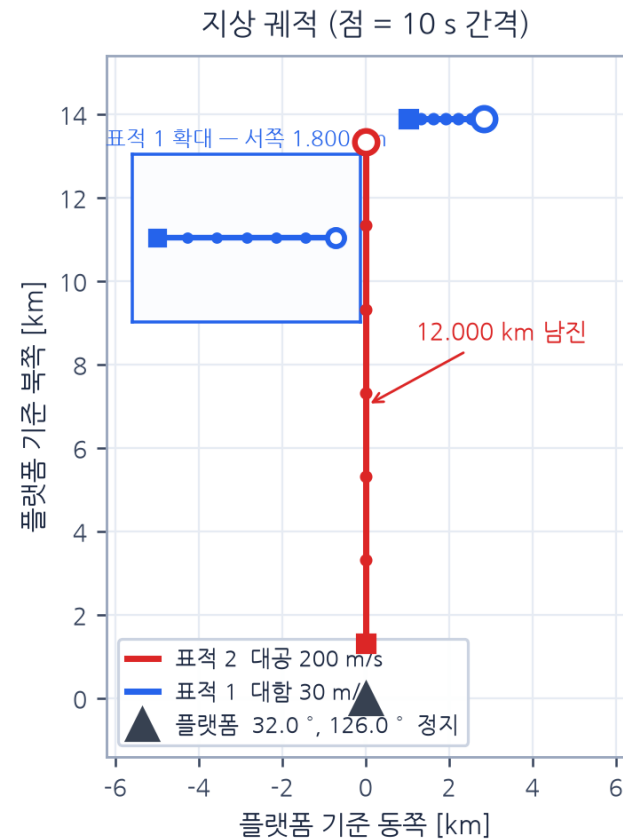
플랫폼 속력이 0 인 점

각속도 계산에 속력이 분모로 들어가지만,
그 계산은 기동 조회 안에만 있다.
플랫폼은 기동이 없어 그 자리를 지나가지 않는다.

03. 시뮬레이션 결과 및 검증

결과 ① — 60 초 궤적

플랫폼을 원점으로 잡은 동 - 북 평면입니다. 점 하나가 10 초입니다.



03. 시뮬레이션 결과 및 검증

결과 ② — 세 가지 방법으로 검증

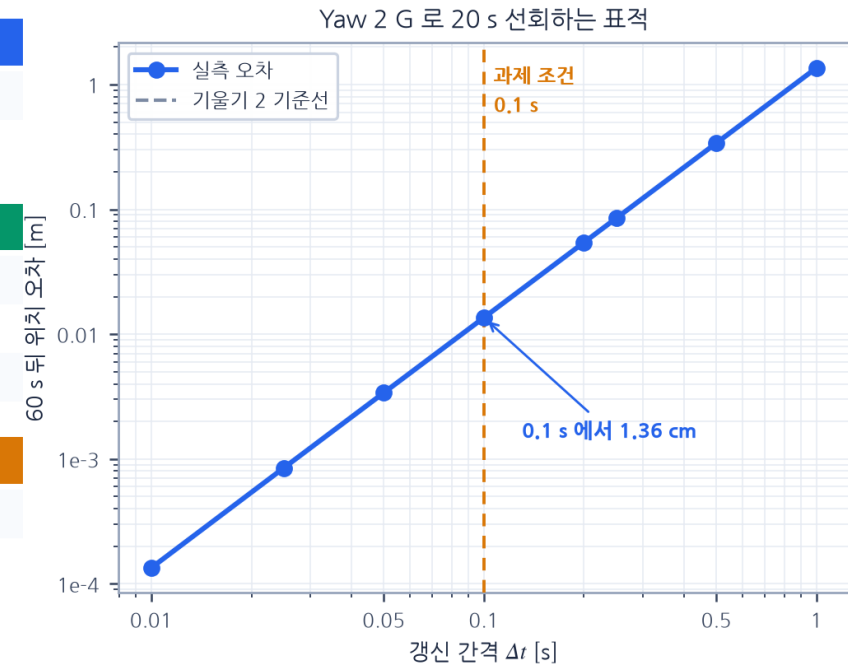
각각이 다른 종류의 실수를 잡습니다 — 스케일, 방향, 그리고 적분 방법.

세 가지 방법으로 검증했다

① 해석해와 대조		
스케일이 틀린 경우를 잡는다		
대함 이동거리	$30 \times 60 = 1800 \text{ m}$	$+1.3\text{e-}08 \text{ m}$
대공 이동거리	$200 \times 60 = 12000 \text{ m}$	$+1.7\text{e-}07 \text{ m}$
② 불변량 보존		
방향이 틀린 경우를 잡는다		
고도 (수평 비행)	300 m 유지	$30.8 \mu\text{m}$
속력 (회전만)	200 m/s 유지	0 (601 표본 일치)
위도 · 경도 (직진)	정서진 · 정남진	$0.1 \mu\text{m} / 0$
③ 수렴 차수		
적분 방법이 틀린 경우를 잡는다		
Δt 를 1/2 로	오차 1/4 (2차)	기울기 2.002

이동거리는 601개 표본 사이 거리를 전부 더해 낸 값이다 — 직선거리가 아니라 실제로 지난간 길이. 12 km 를 날아 잔차가 0.0002 mm 수준이다.

고도 잔차 $30.8 \mu\text{m}$ 는 제공 코드의 ECEF→LLA 반복 근사에서 오는 상수 편차다.



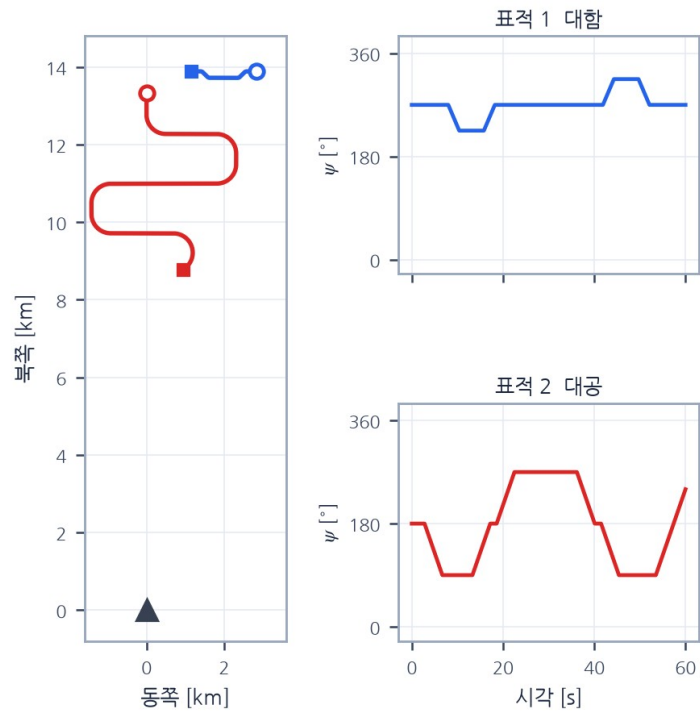
값 하나가 작은 것보다, 간격을 바꿨을 때 정해진 기울기로 줄어드는 것이 더 강한 증거다.

03. 시뮬레이션 결과 및 검증

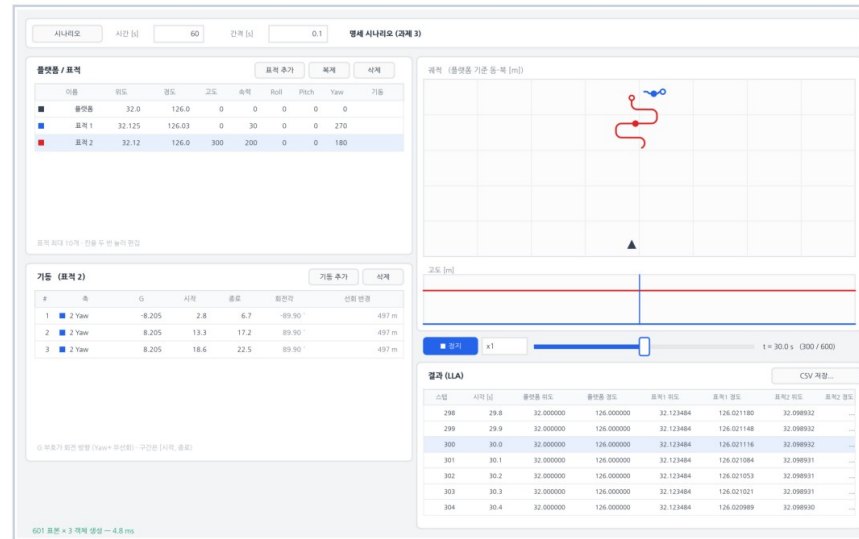
결과 ③ — 기동 시연과 운용 화면

초기값 · 시간 · 간격은 명세 그대로, 기동표 10 줄만 추가했습니다.

기동 10줄을 얻은 60 s 궤적



TargetSim — 값을 바꿔 가며 확인하는 화면



고치면 곧바로 다시 돈다

표의 칸을 고치면 0.25 s 뒤 자동으로 다시 계산한다. 601 표본 × 3 객체가 4.8 ms 라 기다릴 일이 없다.

그림에서 끌어서 배치

시작점과 기수 손잡이를 마우스로 끌면 위도 · 경도 · Yaw 가 표에 반영된다. 회전각과 선회 반경은 G 로부터 계산.

결과는 표 · CSV · 그림

시각별 LLA 를 표로 보고 슬라이더로 되감고 CSV 로 내보낸다. 오늘 숫자가 전부 여기서 나온 CSV 다.

03. 시뮬레이션 결과 및 검증

정리

요구사항과 결과를 한 장에 놓고 보겠습니다.

과제 요구사항	구현	근거
표적 10 · 기동 30	고정 배열로 상한, 범위 밖은 오류 코드	범위 검사
위치 LLA · 속도 V_{heading}	ST_TargetInit, 방향은 자세각이 담당	검증 통과
기동 = $G \cdot \text{축} \cdot \text{시작} \cdot \text{종료}$	ST_TargetManeuver 4 필드, 구간 겹침 검사	검증 통과
ECEF 에서 상태 갱신	위치 · 속도 ECEF, 출력용 LLA 병행 보관	고도 잔차 30 μm
$V_{\text{heading}} \rightarrow V_x, V_y, V_z$ (추가 고민)	동체 $\rightarrow$ NED $\rightarrow$ ECEF, 회전 2 회	속력 완전 보존
제공 좌표변환 코드 사용	CoordinateTransform.c / matrixCalcLib.c 원본	무수정
결과를 LLA 로 출력하고 그림으로	표 · CSV · 궤적 · 고도 · 방위 그림	601 표본 출력

남겨 둔 한계

- Pitch $\pm 90^\circ$ 근처의 짐벌락 — 사원수로 전환하면 해소
- 힘을 풀지 않는 운동학 모델 — 탄도 표적에는 부적합
- 고도 잔차 30.8 μm 는 제공 코드의 ECEF $\rightarrow$ LLA 반복 근사

이어 붙일 수 있는 것

- 궤적에 측정 잡음을 얹으면 그대로 추적 필터 입력
- Part 2 의 좌표변환을 붙이면 안테나 기준 $R \cdot A_z \cdot E_i$
- 표본 생성기를 바꾸면 RCS · 도플러까지 확장 가능

감사합니다

질문 주시면 답변드리겠습니다